Code Documentation

Simon Gueraud — M2 Intern

June 26, 2026

My Master 2 four months internship focused on the optimisation of the cryogenic detectors fiducialization within the TESSERACT and RICOCHET collaborations. I developed several codes for the analysis and treatment of simulations in order to sort hits considering their position in the germanium crystals.
This document gives a deeper documentation of the codes made. All the codes referenced can be found on my personal Git repository `https://www.github.com/siguemon`. *The codes are documented and commented.*

Contents

1 Introduction

If you want...

... to sort hit energies depending of a fiducial volume , you may refer to Sec. 2. This details the class used in every other code made in order to make spectra of the fiducial and surface volume energies.

... to study the density of monosite and multisite events for a given simulation , you may refer to Sec. 3.

... to plot any energy spectra, summing over contributions from different simulations , you may refer to Sec. 4

2 fid_class

This program contains a event class : the main goal is to sort energies over the detector and the fiducial volume defined from the fidcrop and on a single event. The study I realised considered a cylindrical fiducial volume which could be defined from a single length : the so-called fidcrop. This represents the distance between the cylinder surfaces of the crystal and the fiducial volume, in meters.

The irregular number of hits per event requires this approach of sorting event by event. The problem is that simulations may have around 10^6 hits and this sorting process requires some running time. Use of python library numba helps gaining a factor 10^1 in the runtime of such a process, by taking about a second to compile the code at the beginning of the run of your program.

It inputs the x, y, z coordinates (in the lab-frame of reference !) and the deposited hit energies of your event. The use of lab-frame of reference coordinates requires some process to identify the detector and make a correction over its position by adding an offset specific to the detector. The offsets values are for the RUN015 geometry and need to be changed if you want to extend the study to other geometries. The geometry of the RUN015 arranges the detector in a 3×3 array which needs a 30deg. rotation correction (see the make_analysis.py from Sec. 4 for generic use).
You need to change the RICOCHET Simulations SharedCodes/SimulationAnalysis/SimuAnalysis/exe/SimpleAna.cxx in order to get the hit coordinates ("HitX", "HitY", "HitZ") and energy ("Edep") as branches of the SimpleAna simulation analysis code. This modified file is given in the package as src/SimpleAna.cxx.

Once that you have initialized an event, the offsets are automatically made and the energies are sorted in a $3 \times 3 \times 2$ table. The 3×3 dimension is for the detector, while the 2 dimension stands for the (Fiducial, Surface) energies of the event. Such a table can be obtained via use of the fid_class.get() method. The interest of such a process is that you can generalize your analysis to a whole collection of events (typically simulation results) and get the energies sorted into a (#events$\times 3 \times 3 \times 2$) numpy array, as shown in the example below. Of course most of the space of the array is empty, but you can get rid of these null energy values simply by considering specific binning in your histograms (for you energy spectra).

We can express the fiducial volume in function of fidcrop, as given in Eq. 1. This function appears several times in the different codes since working with the fidcrop was easier than with the fiducial volume itself.

$$V_{Fiducial\ Volume} = \frac{height - \texttt{fidcrop}}{height} \times \left(\frac{radius - \texttt{fidcrop}}{radius}\right)^2 \tag{1}$$

You can call this class and use it the following way :

```
import numpy as np #for data usual treatment
import uproot #for siumlation treatment using python (numpy)
import fid_class as C #the class we are talking about

tree = uproot.open('simulation.root:ESO') #open the simulation
(X,Y,Z,E) = (tree[key].array(library='np') for key in ('HitX','HitY','HitZ','Edep')) #
                                                associate to each numpy array a branch of the
                                                root file

tmp = (C.event(x,y,z,e,fidcrop=fidcrop) for x,y,z,e in zip(X,Y,Z,E)) #create a event from
                                                fid_class for each event of the simulation :
                                                sort the coordinates over the defined fidcrop
output = np.array([i.get() for i in tmp]) #get the energies table for each detector and each
                                                 volume (fiducial of surface)
del tmp

... #the rest of your code
```

What needs to be changed

If you want to use the code yourself, you may make sure that you are using RUN015 geometry for analyzing your data. Else, you need to adapt the detector dimensions, detbounds and correction of space-offsets so that the

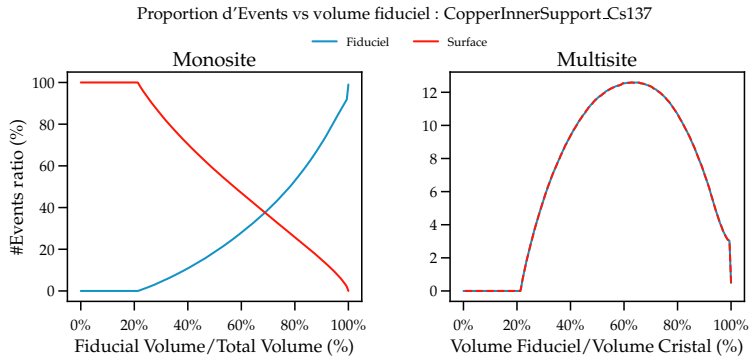
calculation of local coordinates in the detectors frame of reference are right.

3 density_vs_fidcrop

The intended use of these codes is made by using the bash script, itself calling the python program that plots the proportion of monosite and multisite events as a function of the fiducial volume. The usage for this script is the following :

```
$ bash density_vs_fidcrop.sh <simulation filename> <#points=2>
```

Note that the first argument of the bash script (`simulation filename`) is used with a joker (*) and can allow to run every simulation files in the same directory.
You can also change the directories for the python program location, simulation file location, and plot figure saving location. The python program displays the status of the process since this can take a very long time : this uses the `fid_class.py` module (see Sec. 2), and each point requires 3 seconds to run.
An example of the plot saved by the python program is given below :



The folder `density_vs_fidcrop` features examples directories, with a simulation that can be directly analyzed and whose plot can be saved in the directories written in the bash script in order to understand the way this works by running the command given above.

What needs to be changed

If you want to use this code yourself, you'll need to change some paths that appear in the codes, in particular the `LOCAL`, `PRGM`, `SIMUS` and `SAVE` paths in the `density_vs_fidcrop.sh` bash script. The python script `density_vs_fidcrop.py` also calls a `.mplstyle` (matplotlib's custom style file), that changes the appearance of the plot obtained. Technically, you can skip this line and the rest should work fine, but it would probably need a bit of adjustment. Else, the `.mplstyle` file that I call is given in the package.

4 make_total_spectra and plot_total_spectra

These two separate codes are made to compute the total spectra and plot it. The need of separating the two processes is because computing the spectra takes about 40 minutes (4 fiducial cuts $\times$ 300 simulations $\times$ 2 seconds/simulation), before displaying the results in a separate program.

4.1 make_total_spectra

This program opens a simulation, get the associated energy table for `fid_class.py` (cf. Sec. 2), converts the counts to DRU and sums it to the total contribution before carrying on the next simulation. Whenever an error occurs or the program is stopped, the total spectra is saved into a numpy `.npz` file as if the job was done.
You may run the program to obtain an output file via the following command :

```
$ bash make_total_spectra.sh <output filename>
```

The whole process is structured such that the central program `main.py` calls different modules, namely `make_analysis.py`, `conversion.py` and `end_prgm.py` :

`make_analysis.py` features a function `analysis_file()` which takes the path to the desired simulation file, the `fidcrop` mentionned in Sec. 2, and the binning. This function returns the spectra in counts using the binning given in arguments as a numpy array. This module also features a customized exception `NoEvent` for (rare) cases when simulations have no event and thus return an empty array.

`conversion.py` makes the conversion from the counts given by `make_analysis.py` to normalized unit DRU. This requires to access to the contaminated volume and contaminant informations. This module features a main function, `counts_to_dru()` which calls other `get` functions such as `get_nprimaries()`, `get_activity()`, `get_detector_mass()` and `get_source_mass()` which use the different activity measurement files for the materials (that requires the intermediate process of using a mapping from the volume name to the material name), or the total mass files of different volumes of the RUN015 geometry. As for `make_analysis.py`, this module features three custom exceptions such as `NoActivityVolume`, `NoActivityContaminant` and `NoSourceMass`.

`end_prgm.py` is a small module called whenever the data needs to be saved, meaning either at the end of the job or if the user interrupted the program. This features a `stop()` function that is called by the `main.py` program.

The `main.py` uses the described modules to loop over given `fidcrops` and every simulation, and combine the associated process to save the total spectra, meaning the sum of every simulation contribution normalized in DRU. The way it loops over every simulation file is via using `simu_names.json`, that lists every contaminant for every volume in the simulation. It displays the current status in the console and handles the different custom exceptions listed above.

The name of the saved data file is the first argument of the python program when calling it :

```
$ python main.py <output filename>
```

At the very beginning of the program, the program checks if you do want to save data in this file. The data stored into the file is organized as a dictionnary of a numpy array. It contains the bins used in the program, each spectra over the different categories (fiducial monosite : F1, fiducial multisite : F2, surface monosite : S1, surface multisite : S2) and for considered `fidcrop`, with the associated error. The summarized structure of the output datafile is given by the example in Tab. 1.

Key name	Description
`bins`	bins used for the spectra
`F1_1.0mm`	spectra for fiducial monosite events with `fidcrop`=1.0mm
`ERRS2_2.5mm`	error on spectra for surface multisite events with `fidcrop`=2.5mm
...	every other combinations...

Table 1: Generic examples of output `.npz` files keys, with description.

As for Sec. 3, an example is given in the package so that you have every file needed to obtain a single-contribution spectra. This will take every data (either the simulation or the activities, masses files) from the `data` directory, and write the output obtained in the `data/out` directory.

What needs to be changed

The directories in the bash script may be changed to fit your situation. The location of the data files used in the counts to DRU conversion also need to be changed directly in the `conversion.py` module, in the very first lines of the program.

4.2 plot_total_spectra.py

Plotting the total spectra calculated with `make_total_spectra` is achieved by `plot_total_spectra.sh`. This script takes as first argument the name of the `.npz` file outputted by `make_total_spectra.py`, and the place to save the plot of the spectra as second argument.

```
$ bash plot_total_spectra.sh <input filename> <output plot filename>
```

This second programs allows to work on the data obtained without having to make the calculations over again and save a lot of time. An example of the output plot is given in Fig. 1.

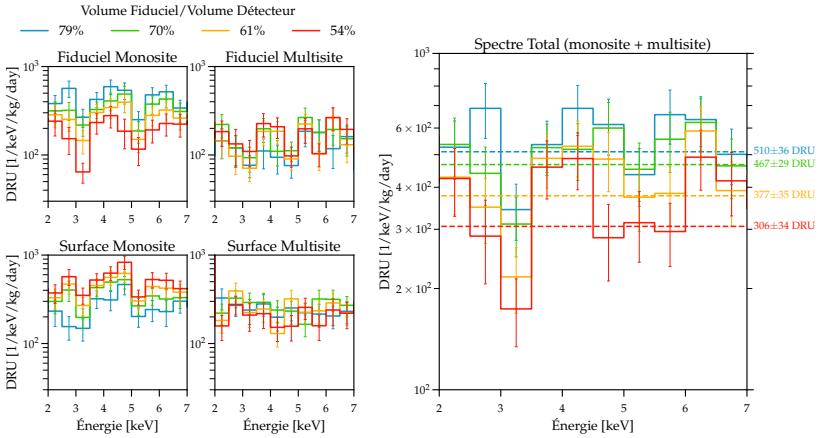


Figure 1: Example of plot given by `plot_total_spectra.py`. The figure structures as each separated spectra on the left side, and the merged multisite+monosite spectra on the right side, with fit lines on the right plot that have been used for the comparison with the experimental data within my internship.

The example given in `make_total_spectra` is continued in this section so that you can understand the way this works and obtain the plot of the spectra for a single contamination.

What needs to be changed

As for Sec. 3, the matplotlib's style file called in the program (line 79) may need to be either removed or its path would need to be updated. Else, every other change are the directories in the bash script, namely the input file (output from `make_total_spectra.sh`) and the plot output location.